

Consumo de servicios web con REST API

Modalidad: Práctica de laboratorio, individual

1. Propósito de la actividad

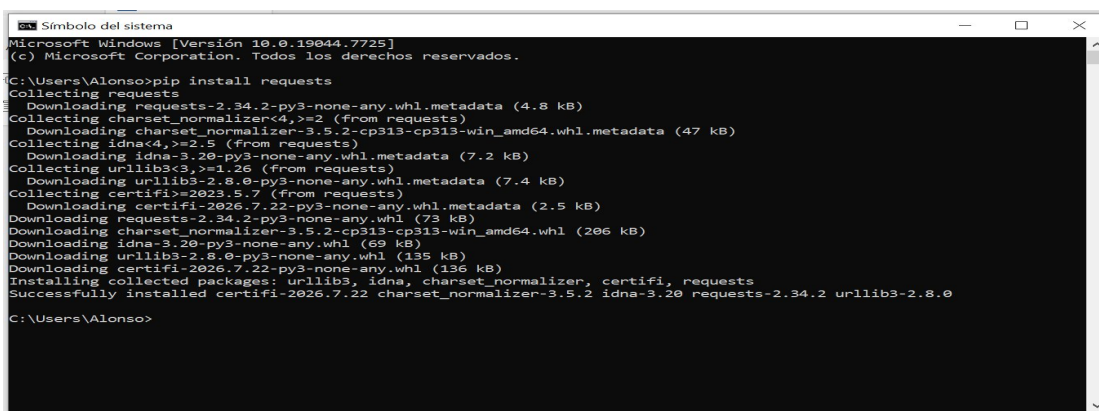
Que el estudiante consuma servicios web a través de una REST API, primero con **Postman** y después con **Python** (lenguaje multiparadigma), interpretando respuestas en formato **JSON** y comparándolas con **XML**, para reafirmar los temas de la unidad:

- APIs y análisis de JSON.
- REST API utilizando Postman.
- REST API utilizando un lenguaje multiparadigma.
- Servicios web con API REST, JSON o XML.

2. Materiales

- Equipo de cómputo con acceso a Internet.
- Postman (aplicación de escritorio o web).
- Python 3 y un IDE (VS Code, PyCharm u otro).
- Librería requests (instalar con `pip install requests`).
- APIs públicas que **no requieren registro ni llave**:
 - o JSONPlaceholder: <https://jsonplaceholder.typicode.com> (API de práctica con recursos posts, users, comments, etc.).
 - o Open-Meteo: <https://api.open-meteo.com/v1/forecast> (pronóstico del clima).

*Nota: JSONPlaceholder **simula** las operaciones de escritura. Responde como si el POST, PUT o DELETE hubieran funcionado, pero los datos no se guardan realmente. Es normal que un GET posterior no muestre el cambio.*



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19044.7725]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Alonso>pip install requests
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.5.2-cp313-cp313-win_amd64.whl.metadata (47 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
  Downloading requests-2.34.2-py3-none-any.whl (73 kB)
  Downloading charset_normalizer-3.5.2-cp313-cp313-win_amd64.whl (206 kB)
  Downloading idna-3.20-py3-none-any.whl (69 kB)
  Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
  Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.2 idna-3.20 requests-2.34.2 urllib3-2.8.0

C:\Users\Alonso>
```

3. Parte A. Repaso conceptual (20 min)

Responde en tu cuaderno o en un documento, con tus propias palabras:

1. ¿Qué es una API y qué significa que una API sea REST?

Una API (Application Programming Interface) es un conjunto de reglas que permite que diferentes aplicaciones se comuniquen entre sí.

- Describe la diferencia entre cliente y servidor en una petición HTTP.

El cliente es quien realiza una petición al servidor, por ejemplo, Postman o un programa escrito en Python.

- Completa la tabla de métodos HTTP:

Método	Operación CRUD	Para qué se usa	Código de éxito típico
GET	Leer	Consultar información	200
POST	Crear	Crear un recurso	201
PUT	Actualizar	Reemplazar un recurso completo	200
PATCH	Actualizar	Modificar parcialmente un recurso	200
Método	Operación CRUD	Para qué se usa	Código de éxito típico
DELETE	Eliminar	Eliminar un recurso	200 o 204

- Explica qué significan los códigos de estado: **200, 201, 204, 400, 401, 404 y 500**.

200: La petición se realizó correctamente.

201: Se creó correctamente un nuevo recurso.

204: La petición fue correcta, pero no hay contenido que devolver.

400: La petición tiene un error.

401: Se requiere autenticación o las credenciales no son válidas.

404: El recurso solicitado no existe.

500: Ocurrió un error interno en el servidor.

- Escribe un objeto JSON que represente a un dispositivo de red con: nombre, dirección IP, modelo, estado (activo o inactivo) y una lista de interfaces. Indica qué tipo de dato es cada valor.

```
{
  "nombre": "Router-Central",
  "direccionIP": "192.168.1.1",
  "modelo": "Cisco ISR 4331",
  "estado": "activo",
  "interfaces": [
    "GigabitEthernet0/0/0",
    "GigabitEthernet0/0/1",
    "Serial0/1/0"
  ]
}
```

4. Parte B. Consumo de la API con Postman (60 min)

Crea una **colección** en Postman llamada Unidad2_Act2_TuNombre y dentro de ella una petición por cada ejercicio. Toma captura de pantalla de la petición y de la respuesta (cuerpo, código de estado y encabezados).

#	Petición	URL	Qué debes observar
B1	GET lista de publicaciones	https://jsonplaceholder.typicode.com/posts	Código 200 y que el cuerpo es una lista (arreglo) de objetos.
B2	GET un recurso	https://jsonplaceholder.typicode.com/posts/1	Campos <code>userId</code> , <code>id</code> , <code>title</code> , <code>body</code> .
B3	GET con parámetros	https://jsonplaceholder.typicode.com/posts?userId=3	Solo regresa publicaciones del usuario 3. Anota cuántas son.
B4	GET recurso anidado	https://jsonplaceholder.typicode.com/posts/1/comments	Comentarios de la publicación 1.
B5	POST crear	https://jsonplaceholder.typicode.com/posts	En <i>Body</i> → <i>raw</i> → <i>JSON</i> envía un objeto con <code>title</code> , <code>body</code> y <code>userId</code> . Código 201 y un <code>id</code> nuevo.
B6	PUT reemplazar	https://jsonplaceholder.typicode.com/posts/1	Envía un objeto completo. Compara con B2.
B7	PATCH modificar	https://jsonplaceholder.typicode.com/posts/1	Envía solo <code>{"title": "Nuevo título"}</code> .
B8	DELETE eliminar	https://jsonplaceholder.typicode.com/posts/1	Código 200 y cuerpo vacío <code>{}</code> .
B9	GET recurso inexistente	https://jsonplaceholder.typicode.com/posts/9999	Código 404. Explica qué significa.
B10	GET con parámetros de consulta	https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-	Temperatura y viento actuales de Querétaro.
#	Petición	URL	Qué debes observar
		100.39&current=temperature_2m,wind_speed_10m	Identifica las unidades en el campo <code>current_units</code> .

B1

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B1

GET <https://jsonplaceholder.typicode.com/posts> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

200 OK • 421 ms • 7.77 KB

Save and document request B1

Use B1 vars from your env

Create a follow-up request chain

1

Auto

B2

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B2

GET <https://jsonplaceholder.typicode.com/posts/1> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

200 OK • 259 ms • 1.09 KB

Set B2 to use baseUrl var

Add a 404/invalid-id test

Set up a Postman Monitor

1

Auto

B3

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B3

GET <https://jsonplaceholder.typicode.com/posts?userId=3> Send

Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ userId	3			
Key	Value	Description		

Body [Visualize](#) [Save Response](#) ...

```
{
  "userId": 3,
  "id": 21,
  "title": "asperiores ea ipsam voluptatibus modi minima quia sint",
  "body": "repellat aliquid praesentium dolorem quo nse totam minus non itaque n nihil labore molestiae sunt dolor eveniet hic recusandae veniam ntempora et tenetur expedita sunt"
}
```

200 OK • 286 ms • 1.86 KB

Local mode Terminal Console The coast is clear

Globals Vault Tools

New Chat

Make B3 use shared environment vars
Clone B3 into a clear CRUD set
Write a README for the B-series

1

Auto

B4

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B4

GET <https://jsonplaceholder.typicode.com/posts/1/comments> Send

Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body [Visualize](#) [Save Response](#) ...

```
{
  "postId": 1,
  "id": 1,
  "name": "id labore ex et quam laborum",
  "email": "Eliseo@gardner.biz",
  "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos ntempora quo necessitatibus ndolor quam autem quasi nreiciendis et nam sapiente accusantium"
}
```

200 OK • 95 ms • 1.53 KB

Local mode Terminal Console The coast is clear

Globals Vault Tools

New Chat

Add auth + headers baseline
Define example responses & tests
Organize into a CRUD folder set

1

Auto

B5

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase. [Try Native Git](#)

Unidad2_Actividad2_Alonso > B5

POST <https://jsonplaceholder.typicode.com/posts> [Send](#)

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body [Visualize](#)

201 Created • 337 ms • 1.09 KB

```
{
  "title": "Prueba de red",
  "body": "Contenido de ejemplo",
  "userId": 1,
  "id": 101
}
```

Save and parameterize post id
Add negative-case assertions
Create a matching GET/verify flow

1

Auto

B6

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase. [Try Native Git](#)

Unidad2_Actividad2_Alonso > B6

PUT <https://jsonplaceholder.typicode.com/posts/1> [Send](#)

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON [Schema](#) [Beautify](#)

1 {
2 "id": 1,
3 "title": "Prueba de red",
4 "body": "Contenido actualizado",
5 "userId": 1
6 }

Body [Visualize](#)

200 OK • 123 ms • 984 B

```
{
  "id": 1,
  "title": "Prueba de red",
  "body": "Contenido actualizado",
  "userId": 1
}
```

Convert it to a reusable flow
Add negative-case tests
Chain POST → GET verification

1

Auto

B7

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B7

PATCH https://jsonplaceholder.typicode.com/posts/1

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

none

This request does not have a body

Body {} JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

200 OK • 111 ms • 1.07 KB

Save Response

Fix B7 URL & rerun test loop

Add Tests for status & body

Create env var for postId

1

Auto

B8

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B8

DELETE https://jsonplaceholder.typicode.com/posts/1

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body {} JSON Preview Visualize

```
1 {}
```

200 OK • 256 ms • 896 B

Save Response

Fix B8 DELETE URL & vars

Add 404 assertion for B8

Create CRUD runner with B8

1

Auto

B9

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B9

GET `https://jsonplaceholder.typicode.com/posts/9999` Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body `{}` JSON Preview Debug with AI

1 `{}`

Investigate 404 error

404 Not Found • 241 ms • 908 B

Diagnose Decline

New Chat

Investigate 404 error

→ The request is returning a 404 Not Found error because the post ID 9999 does not exist on the server. I will hand this over to Agent Mode so you can investigate the available resources or adjust the request to target a valid post ID.

Think Harder

Define variables and base URL

Add tests for success and schema

Wire B9 into the CRUD runner

1

Auto

B10

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Actividad2_Alonso > B10

GET `https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-100.39¤t=temperature_2m,wind_speed_10m` Send

Query Params

Key	Value	Description	Bulk Edit
latitude	20.59		
longitude	-100.39		
current	temperature_2m,wind_speed_10m		

Body `{}` JSON Preview Visualize

1 `{`

2 `"latitude": 20.56239,`

3 `"longitude": -100.43347,`

4 `"generationtime_ms": 0.08106231689453125,`

5 `"utc_offset_seconds": 0,`

6 `"timezone": "GMT",`

7 `"timezone_abbreviation": "GMT",`

8 `"elevation": 1821.0,`

9 `"current_units": {`

10 `"temperature": "Celsius",`

11 `"wind_speed": "m/s"`

12 `}`

13 `}`

200 OK • 1.20 s • 427 B

Save Response

Fix URL params and verify parsing

Add a parameterized lat/long run

Refactor into a shared request builder

1

Auto

Preguntas de la Parte B

- ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía?
PUT envía normalmente el recurso completo para reemplazarlo.
PATCH solamente envía los campos que se quieren modificar.
- ¿Qué encabezado Content-Type regresa el servidor y qué indica?
Content-Type: application/json; charset=utf-8
- En B10, ¿qué parte de la URL es la ruta y cuál son los parámetros de consulta (query params)?
https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-100.39¤t=temperature_2m,wind_speed_10m
Ruta: /v1/forecast

Query params: latitude=20.59
longitude=-100.39
current=temperature_2m,wind_speed_10m

5. Parte C. Consumo de la API con Python

Crea una carpeta `actividad_rest` y un archivo `cliente_rest.py` por cada ejercicio

C1. Primer GET

```
import requests

BASE="https://jsonplaceholder.typicode.com"

respuesta=requests.get(f"{BASE}/posts/1",timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipodecontenido:",respuesta.headers["Content-Type"])

publicacion = respuesta.json() #convierteelJSONenundiccionarioidePython
print("Título:", publicacion["title"])
```

C2. Recorrer una lista y usar parámetros

```
respuesta=requests.get(f"{BASE}/posts",params={"userId":3},timeout=10)
publicaciones = respuesta.json() # lista de diccionarios

print("Total:",len(publicaciones))
for p in publicaciones:
    print(f"{p['id']:>3}|{p['title']}")
```

C3. POST, PUT y DELETE

```
nuevo={"title":"Pruebadered","body":"Contenidodeejemplo","userId":1}

r=requests.post(f"{BASE}/posts",json=nuevo,timeout=10)
print("POST:", r.status_code, r.json())

r=requests.put(f"{BASE}/posts/1",json={**nuevo,"id":1},timeout=10)
print("PUT:", r.status_code, r.json())

r=requests.delete(f"{BASE}/posts/1",timeout=10)
print("DELETE:", r.status_code)
```

C4. Manejo de errores

Modifica C1 para que consulte `/posts/9999` y maneje el error correctamente:

```

try:
    r=requests.get(f"{BASE}/posts/9999",timeout=10)
    r.raise_for_status()

    #lanzaexcepti
    ón si el código es 4xx o 5xx print(r.json())
except requests.exceptions.HTTPError as e: print("Error
HTTP:", e)
except requests.exceptions.RequestException as e:

```

C5. Consumir otra API (clima)

Escribe una función `obtener_clima(latitud, longitud)` que consulte Open-Meteo y regrese un diccionario con la temperatura y la velocidad del viento. Usa `params` en lugar de armar la URL a mano.

```

def obtener_clima(latitud, longitud):
    url="https://api.open-meteo.com/v1/forecast"
    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m, wind_speed_10m",
    }
    r=requests.get(url, params=parametros, timeout=10)
    r.raise_for_status()
    return r.json()["current"]

```

Probarla con tres ciudades distintas (por ejemplo Querétaro: 20.59, -100.39) y mostrar los resultados en una tabla impresa en consola.

C1

```

C:\Users\Alonso>python c1.py
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Escogido Lara Jorge Alonso

```

C2

```

C:\Users\Alonso>python c2.py
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
Escogido Lara Jorge Alonso

C:\Users\Alonso>

```

C3

```
C:\Users\Alonso>python c3.py
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
Escogido Lara Jorge Alonso
```

C4

```
C:\Users\Alonso>python c4.py
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999
Escogido Lara Jorge Alonso
```

C5

```
C:\Users\Alonso>python c5.py
"
Ciudad          | Temp (°C)  | Viento (km/h)
"-----"
"Querétaro      | 24.5       | 15.3
Ciudad de México | 22.1       | 5.1
Guadalajara     | 25.8       | 8.2
"
(Escogido Lara Jorge Alonso
```

6. Parte D. JSON frente a XML (30 min)

1. Toma la respuesta JSON de `/posts/1` y escribe a mano su equivalente en XML.
2. Con Python, convierte el diccionario a XML usando `xml.etree.ElementTree`:

```
import xml.etree.ElementTree as ET

raiz=ET.Element("publicacion")
for clave, valor in publicacion.items():
    ET.SubElement(raiz,clave).text=str(valor)

print(ET.tostring(raiz,encoding="unicode"))
```

3. Completa la tabla comparativa:

Criterio	JSON	XML
Legibilidad		
Tamaño del mensaje		
Tipos de datos		
Uso común en APIs REST		

7. Parte E. Mini proyecto integrador (60 min)

Desarrolla un script `reporte_api.py` que:

1. Consulte `/users` de JSONPlaceholder y obtenga el nombre, correo y ciudad (`address.city`) de cada usuario.
2. Para cada usuario, consulte cuántas publicaciones tiene (`/posts?userId=<id>`).
3. Consulte el clima actual de un punto geográfico de tu elección.
4. Maneje errores de conexión y de código de estado.
5. Guarde el resultado en un archivo `reporte.json` (usa `json.dump` con `indent=4` y `ensure_ascii=False`) y también imprima un resumen en pantalla.

Estructura sugerida del archivo de salida:

```
{
  "clima":{"temperature_2m":0.0,"wind_speed_10m":0.0}, "usuarios": [
    {"nombre":"...", "correo":"...", "ciudad":"...", "publicaciones":0}
  ]
}
```

8. Entregables (portafolio de evidencias)

1. Documento con las respuestas de la Parte A, las preguntas de la Parte B y la tabla de la Parte D.
2. Exportación de la colección de Postman (archivo `.json`) y capturas de pantalla de B1 a B10.
3. Archivos `.py` de las Partes C y E, con comentarios.
4. Archivo `reporte.json` generado por el mini proyecto.
5. Conclusión breve (5 a 8 líneas): ¿en qué casos de administración de redes serviría consumir una API REST?

9. Lista de verificación para la evaluación

Criterio	Cumple	No cumple
Identifica correctamente métodos HTTP y códigos de estado		
Ejecuta las 10 peticiones en Postman con evidencia		
Envía cuerpo JSON y parámetros de consulta correctamente		
Consume la API con Python usando <code>requests</code>		
Convierte respuestas JSON a estructuras de Python (diccionarios y listas)		

Criterio	Cumple	No cumple
Maneja errores y tiempos de espera (timeout, raise_for_status)		
Compara JSON y XML con un ejemplo funcional		
El mini proyecto genera el archivo reporte.json correcto		
Código ordenado, comentado y entregado a tiempo		